



Hooks, guardrails, евалуације и завршни QA

Извршиве политике и докази за контролисан AI развојни ток

Инструкција није довољна када је правило обавезно

AI инструкција може рећи „не читај .env”, али обавезно правило треба спровести кодом када је могуће.

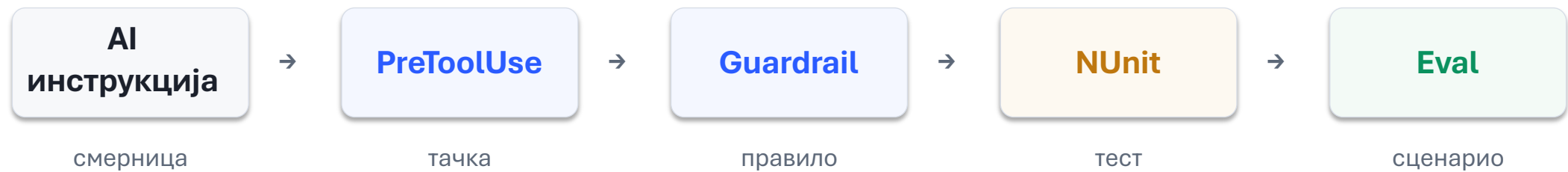
Зато solution садржи EquipmentReservation.Guardrails и NUnit тестове његових политика.

КЉУЧНА ПОРУКА

Guardrail је извршива заштита око агентског тока.

Heuristika + deterministička zaštita

Заштита се поставља пре позива алата, током извршења и пре завршног прихватања.



КЉУЧНА ПОРУКА

Правила се не остављају само на добро понашање модела.

Guardrail је интерфејс, не велики if блок

OCP и DIP важе и за развојни tooling.

GuardrailEvaluator зависи од колекције апстракција, па се нова политика додаје новом класом уместо проширивањем централног условног израза.

КЉУЧНА ПОРУКА

То је исти дизајнерски принцип као у пословном коду.

IToolGuardrail и GuardrailEvaluator

Мала проверљива правила лакше је тестирати и проширити.

КОД

```
1 public interface IToolGuardrail
2 {
3     GuardrailDecision Evaluate(ToolInvocation invocation);
4 }
5
6 public sealed class GuardrailEvaluator(IEnumerable<IToolGuardrail> guardrails)
7 {
8     {
9         public GuardrailDecision Evaluate(ToolInvocation
10             invocation)
11         {
12             foreach (var guardrail in guardrails)
13                 if (!guardrail.Evaluate(invocation).Allowed) return
14                     decision;
15             return GuardrailDecision.Allow();
16         }
17     }
```

Шта је важно

Evaluator зависи од апстракције.

Нова политика је нова класа.

Централни ток остаје стабилан.

Конкретне политике су мале и намерне

Заштита треба да има јасан домет.

01

SensitiveFile

блокира .env и secrets.json

02

DangerousCommand

блокира force push и rm -rf

03

Allowlist

још јача граница за ризичне системе

04

Decision

стабилно Allow/Block понашање

SensitiveFileGuardrail

Правило блокира приступ познатим тајним фајловима.

КОД

```
1 public sealed class SensitiveFileGuardrail :  
2     IToolGuardrail  
3 {  
4     private static readonly string[] ForbiddenNames =  
5         [".env", "secrets.json", "appsettings.secrets.json"];  
6  
7     public GuardrailDecision Evaluate(ToolInvocation  
8         invocation)  
9     {  
10         var normalized = invocation.FilePath?.Replace("\\",  
11             "/");  
12         return ForbiddenNames.Any(name => normalized?.  
13             EndsWith(name) == true)  
14             ? GuardrailDecision.Block("Sensitive file blocked.")  
15             : GuardrailDecision.Allow();  
16     }  
17 }
```

Шта је важно

Тајне не смеју постати context.

Путања се нормализује.

Одлука је детерминистичка.

DangerousCommandGuardrail

Деструктивне команде се блокирају пре извршења.

```
1 private static readonly string[] ForbiddenFragments =  
2 [  
3     "git push --force",  
4     "git push -f",  
5     "rm -rf",  
6     "Remove-Item -Recurse -Force",  
7     "format c:"  
8 ];
```

Шта је важно

Ово је denylist приступ.

За ризичније системе allowlist је јача граница.

Политика мора бити тестирана.

Hook повезује AI алат са политиком

PreToolUse је адаптер за специфичан алат.

Сама guardrail апликација остаје обичан .NET пројекат у solution-у и може се повезати са другим клијентом другим adapter-ом.

КЉУЧНА ПОРУКА

Адаптер се мења, политика остаје.

.claude/settings.json kao adapter

Hook позива guardrail пројекат пре ризног алата.

КОД

```
1 {
2   "hooks": {
3     "PreToolUse": [
4       {
5         "matcher": "Bash|Read|Edit|Write",
6         "hooks": [{
7           "type": "command",
8           "command": "dotnet run --project
9             src/EquipmentReservation.Guardrails/Equipment
10             Reservation.Guardrails.csproj"
11         }]
12       }
13     ]
14   }
15 }
```

Шта је важно

Конфигурација је спољни детаљ.

Политика остаје тестабилна.

Исти принцип граница важи као у Clean Architecture.

Guardrail се тестира као обичан код

Без тестова guardrail је само још једна претпоставка.

Исти EquipmentReservation.Tests пројекат проверава да ризичне команде и приступ .env датотеци буду одбијени.

КЉУЧНА ПОРУКА

Заштита мора бити доказива.

Тестови за guardrail политике

Unit тестови доказују да забране заиста важе.

```
1 [TestCase("git push --force origin main")]
2 [TestCase("rm -rf ./src")]
3 public void DangerousCommandGuardrail_BlocksDestructiveComm
4     ands(string command)
5 {
6     var guardrail = new DangerousCommandGuardrail();
7     var result = guardrail.Evaluate(new ToolInvocation(
8         "Bash", command, null));
9     Assert.That(result.Allowed, Is.False);
10 }
11
12 [Test]
13 public void SensitiveFileGuardrail_BlocksEnvFile()
14 {
15     var result = guardrail.Evaluate(new ToolInvocation(
16         "Read", null, "/repo/.env"));
17     Assert.That(result.Allowed, Is.False);
18 }
```

Шта је важно

Тестови су брзи и детерминистички.

Политика се проверава без AI модела.

Ово је нормалан code quality механизам.

Једна команда проверава пословне и guardrail тестове

Заштита није одвојена од развојног процеса.



```
1 dotnet test EquipmentReservation.sln --configuration
2 Release
```

Шта је важно

Исти solution је извор истине.

Пословни и tooling тестови пролазе заједно.

CI може да проверава обе врсте правила.

Еваулациони сценарији нису исто што и unit тестови

Unit тест доказује једну детерминистичку политику.

Eval сценарио проверава цело агентско понашање у условима који личе на стварни ток рада: да ли модел поштује инструкције, користи алате правилно и не тврди непроверене резултате.

КЉУЧНА ПОРУКА

Оба нивоа су потребна.

Unit тест и eval имају различиту сврху

Не треба једно користити као замену за друго.

Провера	Шта доказује
Unit тест	конкретна политика враћа Allow/Block како је дефинисано
Integration test	solution пролази кроз стварне команде
Eval scenario	agentski ток поштује контекст, ограничења и доказе
Human review	тим прихвата или одбија резултат према критеријумима

Завршни QA гледа више сигнала

Коначна одбрана није једна команда.

01

Архитектура

границе и dependency rule

02

Тестови

пословни и guardrail сценарији

03

AI trag

AI_USAGE и одлуке тима

04

Diff

мали, објашњив и релевантан

Шта није прихватљиво

Guardrails не служе да сакрију неразумевање.

ЛОШЕ

model тврди да је тестирао без команде

чита .env или secrets

force push у дељеном току

мења тест да сакрије грешку

ДОБРО

реална build/test провера

блокирани ризични алати

јасан AI_USAGE запис

review пре прихватања

Рад на вежби

Циљ је да се правило из инструкције претвори у извршиву заштиту.

- 01 Покренути guardrail тестове.
- 02 Проверити да .env приступ буде блокиран.
- 03 Проверити да деструктивне команде буду блокиране.
- 04 Објаснити разлику између hook adapter-а и саме политике.
- 05 Осмислити један eval сценарио за agentski ток.

Суштина Вежбе 8

Одговорна употреба AI алата тражи више од доброг prompt-а.

Када је правило важно, треба га претворити у код, тестирати као сваки други део система и укључити у завршни QA ток.

КЉУЧНА ПОРУКА

То затвара причу предмета.

За завршну одбрану

Студент треба да може да повеже све теме у једну причу.

- 01 Показати захтев, backlog ставку и критеријуме.
- 02 Објаснити архитектонске границе и use-case ток.
- 03 Показати тестове и резултат покретања.
- 04 Показати AI_USAGE и контролисан AI workflow.
- 05 Објаснити MCP/guardrail део без мешања са пословним језгром.